



FPGA Acceleration and Optimization of the CLUE Reconstruction Algorithm using SYCL and Alpaka

Ghala Buarish

Project Partner: Farid Abi Doumit

Supervisors: Andrea Bocci, Mario Gonzalez Carpintero

August 21, 2026

Abstract

This paper investigates the FPGA acceleration of the CLUE clustering algorithm for the upgraded CMS High Granularity Calorimeter (HGCAL). This work builds off a previous SYCL FPGA implementation by identifying the performance bottlenecks and introducing modifications to optimize the implementation. The main focus for optimization was lowering the initiation interval and latency. Additionally, this project adapts the CLUE algorithm to target FPGA execution through Alpaka. The results of the project reflect the potential to improve CLUE throughput through using FPGA hardware.

Contents

1 Introduction.....	3
1.1 CMS and the HL-LHC.....	3
1.2 Motivation and Objectives.....	3
2 Theoretical Background.....	4
2.1 CLUE Reconstruction Algorithm.....	4
2.2 FPGAs.....	5
2.3 SYCL and Intel oneAPI.....	6
2.4 Alpaka.....	6
3 Methodology.....	7
3.1 Target FPGA Platform : Terasic DE10-Agilex.....	7
3.2 FPGA Development and Evaluation Flow.....	7
3.2.1 SYCL Implementation and Workflow.....	7
3.2.2 Alpaka Implementation.....	8
3.3 Reference Implementations.....	8
3.4 Datasets.....	8
4 SYCL FPGA CLUE Implementation.....	9
4.1 Implementation.....	9
4.1.1 Histogram Modification.....	10
4.1.2 Cluster Assignment Modification.....	11
4.2 Pipelining and Parallelization.....	12
5 Alpaka FPGA CLUE Adaptation.....	12
5.1 Implementation.....	12
6 Results.....	13
6.1 SYCL-Based Implementation Results.....	14
6.1.1 Results Verification.....	14
6.1.2 Performance on FPGA.....	14
6.2 Alpaka FPGA CLUE Adaptation Results.....	15
6.2.1 Results Verification.....	15
7 Conclusions.....	16
7.1 Challenges and Limitations.....	16
7.3 Future Work.....	17
7.4 Acknowledgements.....	17
8 References.....	18

1 Introduction

1.1 CMS and the HL-LHC

The Large Hadron Collider (LHC) is a circular particle accelerator with a circumference of 27 km that accelerates protons or ions to near the speed of light. Two high-energy particle beams circulate inside the accelerator in opposite directions. The beams collide at four locations: CMS, ATLAS, ALICE, and LHCb, where particles in the beams collide. In June of this year, the LHC entered Long Shutdown 3. During this shutdown the LHC accelerator and its experiments are being upgraded in preparation for the High-Luminosity LHC (HL-LHC). The HL-LHC purpose is to increase the intensity of the beam, and thus the number of collisions, allowing more data to be collected for analysis. One of the major upgrades being carried out during this shutdown is the upgrade of the compact Muon Solenoid (CMS) experiment to install the new High Granularity Calorimeter (HGCAL).

CMS is one of two general purpose detectors at the LHC, the other is ATLAS. The CMS detector consists of several layers, each layer is designed to stop, track, or measure a different type of particle emerging from the collisions. Charged particles from collisions are first met by a tracker that charts their positions as they move. Outside the tracker are calorimeters that measure the energy of the particles. To maintain its performance at the HL-LHC, the existing endcap calorimeters will be replaced with the HGCAL. HGCAL is designed to detect particles by providing a fine image of electromagnetic and hadronic showers enabled by its high lateral and longitudinal readout granularity [1]. To process the large data scale emerging from HGCAL in real-time during data taking, a highly efficient clustering algorithm is required. An algorithm developed for this is CLUE [2].

1.2 Motivation and Objectives

HL-LHC and HGCAL will increase computational demands considering the significant data volumes. This motivates investigating hardware accelerators that offer high throughput, particularly FPGAs. FPGAs are investigated here as an alternative or complement to GPUs for executing the CLUE algorithm, a parallel clustering algorithm for high granularity calorimeter reconstruction.

This project investigates the FPGA implementation of CLUE reconstruction algorithm using Intel oneAPI SYCL and Alpaka. The work focuses on verifying the results of the implementations, identifying performance bottlenecks, optimizing SYCL implementation, and investigating the adaptation of the Alpaka implementation for FPGA execution.

2 Theoretical Background

Section 2 presents the theoretical background required for this project; this section covers relevant information about CLUE reconstruction algorithm, FPGA architecture, Intel oneAPI SYCL, and Alpaka.

2.1 CLUE Reconstruction Algorithm

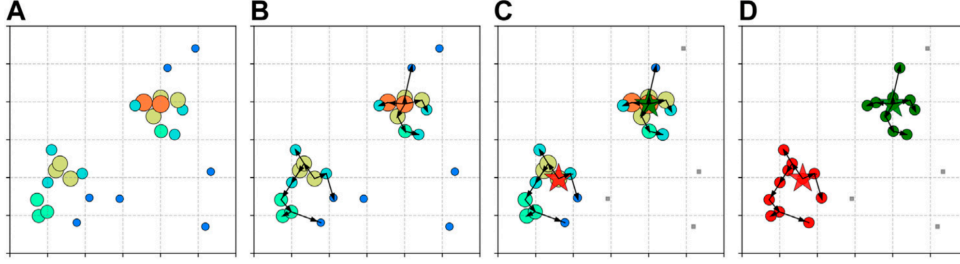


Fig. 1. Demonstration of the CLUE Algorithm [2].

CLUE, CLUestring of Energy, [2] is a proposed algorithm in efforts to handle the high surge in the computing load when clustering digitized energy deposits in the reconstruction stage of CMS high granularity calorimeters.

In particle showers, the sensors on the layer of the shower register the deposited energy as the detector hits. A clustering algorithm is then applied to the data collected by the detectors to group together the hits. The CLUE algorithm processes the data layer by layer which allows it to handle the large number of hits efficiently. It does that by using a fixed-grid and a spatial index to access data points for each layer of the calorimeter. The fixed-grid limits neighborhood search to nearby tiles rather than scanning over all points. Energy deposits are taken as a 2D data point with an associated weight equaling its energy value.

CLUE requires four parameters: ρ_c is the minimum density to promote a point as a seed, d_c is the cut-off distance in the calculation of local density, δ_c and δ_o are the minimum separation requirements for seeds and outliers, respectively.

For each data point CLUE calculates its local density (ρ) and its nearest point with higher density (nh) referred to as the nearest-higher. The local density is defined as the sum of the convolution kernel, $\chi(d_{ij})$, multiplied by the weight of point w_j .

$$\rho = \sum_{j: d_{ij} \leq d_c} \chi(d_{ij}) w_j \quad (1)$$

In the datasets used in this project, all points are assigned an equal weight of $w = 1$. Therefore, ρ depends solely on the spatial distribution of neighboring points.

CLUE determines the nearest-higher of point i by the distance from point i to its nearest point with higher ρ . This separation is defined as δ . If there is no higher density point within the search distance, CLUE defines $nh_i = -1$ and $\delta_i = +\infty$. Points with $\rho_i > \rho_c$ and $\delta_i > \delta_o$ are classified as cluster seeds. Points with $\rho_i < \rho_c$ and $\delta_i > \delta_o$ are classified as outliers. Outliers are noise that is rejected by the CLUE algorithm through this classification process.

A point is set as a follower to its nearest-higher; each point would have a list of followers. Clusters are then assigned by iterating over the followers of each seed; this enables parallelization since each list of followers is independent from another.

Figure 1 breaks down the CLUE algorithm into four steps. Step A is calculating the local density for each point using Eq. 1. Colors and sizes represent different local densities. In step B, CLUE calculates the nearest-higher and the separation for each point. Step C, points are promoted as seeds if ρ and δ are large enough and marked with a star; points with small ρ and large δ are demoted to outliers and are greyed out. In the last step, CLUE assigns the cluster indices from seeds through their chain of followers; outliers receive a cluster id of -1.

The steps demonstrated in Fig. 1 are mapped to computational stages in the accelerator implementations of CLUE.

2.2 FPGAs

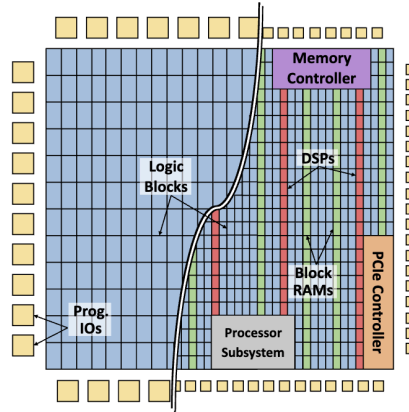


Fig. 2. FPGA Architecture [3].

Field-programmable gate arrays (FPGAs) consist of programmable blocks connected together using wires with programmable switches between them. The connectivity of routing switches are controlled using configuration bits stored in static random access memory (SRAM) cells. FPGAs can implement the exact hardware needed for each application rather than a fixed instruction-set architecture like that of general-purpose processors (CPUs) or graphics processing units (GPUs). While a custom system-on-chip (SoC) device would provide a high level of customization and thus

a high performance, it is more expensive and time consuming to implement than an FPGA. Furthermore, an algorithm implemented on a SoC would provide no flexibility for future modifications. An illustration of the architectural components of an FPGA is shown in Fig. 2.

Programmable blocks are made up of flip-flops (FFs) and lookup tables (LUTs). Flip-flops are single bit binary shift registers used to synchronize logic and store logical states. To implement different Boolean expressions in the blocks, FPGAs use SRAM-based lookup tables, LUTs. combinatorial logic like ANDs, ORs, and NANDs are implemented as truth tables within LUT memory. An LUT can implement K-input Boolean function by storing its truth table in 2^K configuration SRAM cells, as a result, the area of the LUT increases exponentially with K and its speed degrades linearly; this creates an area-delay tradeoff [3].

Multiplying numbers together can become expensive when using combinatorial logic, hence FPGAs have prebuilt multiplier circuitry referred to as digital signal processors (DSPs) to minimize LUT and flip-flop usage.

Another resource of FPGAs is block RAMs. block RAMs consist of a two dimensional array of SRAM cells and peripheral circuitry to access the cells for read or write operations. FPGA RAMs have configurable width and depth allowing them to adapt to application needs. FPGAs also include configurable input/output (I/O) resources that provide the interface between internal FPGA architecture and external devices.

2.3 SYCL and Intel oneAPI

SYCL is a high-level C++ programming model that allows data parallelism for different heterogeneous systems [4]. Device code is expressed as kernels and is submitted through a queue for execution on a selected device. The kernel code becomes the RTL IP core while the host code runs on the CPU and serves as the testbench. Both the device code and host code can be contained in the same C++ source.

SYCL is supported through Intel oneAPI DPC++/C++ compiler. The compiler offers several compilation stages that allow for debugging, verification, and evaluation before hardware execution. The functionality of the algorithm is verified by compiling the device code to execute on a CPU. The compiler generates an FPGA optimization report that provides information about the performance bottlenecks and hardware structure. When an FPGA device is selected, the compiler builds a custom hardware datapath in the FPGA's logic. The different stages of FPGA testing and compilation are discussed in further detail in section 3.2.

2.4 Alpaka

Alpaka is an abstraction library that provides performance portability across various accelerators [5]. Alpaka library provides multiple accelerator back-end variants which can be specified at

runtime. Similar to CUDA, Alpaka uses grid-block-threads domain decomposition strategy. Parallelization is done by dividing the algorithm into a multi-dimensional grid of work items referred to as kernels. Blocks are executed independently, while threads in a block are executed in parallel. Threads in the same block can communicate and synchronize using shared memory local to that block. However, global memory is accessible by all threads in the grid.

3 Methodology

This section explains the target FPGA platform, development and evaluation workflow, reference implementations, and the datasets used throughout the project.

3.1 Target FPGA Platform : Terasic DE10-Agilex

In this paper, all versions of the CLUE algorithm evaluated on the simulation stage were tested using the Terasic DE10-Agilex Development and Education Kit [6], featuring an Altera Agilex FPGA. The board is supported by the Intel oneAPI toolkits. This paper builds on previous work in which the algorithm was tested on a BittWare IA-840 board featuring an Altera Agilex FPGA as well.

3.2 FPGA Development and Evaluation Flow

3.2.1 SYCL Implementation and Workflow

The Intel oneAPI FPGA development flow starts with creating the SYCL kernel and host code, which are combined to form a multiarchitecture binary for the FPGA acceleration flow. The first stage after creating the SYCL C++ code is to verify correctness. This is done through the Intel FPGA Emulation Platform for OpenCL software. It executes the device code on the CPU. During the Emulation stage, performance is not representative of that of the FPGA since it is not targeting a specific hardware [4].

The next stage is compiling for the FPGA Simulator. To compile for the simulator, the Board Support Package (BSP) for the target FPGA board needs to be installed. After compilation is completed, an FPGA optimization report is generated containing information about the kernels transformed into an FPGA design. The report includes performance metrics, expected performance bottlenecks, and estimated resource utilization. When running the simulation executable, the report would include a summary of the simulated performance results as well. During this stage, the compiler generates RTL files that could be simulated on the Questa Intel FPGA Edition software. The simulation reflects the exact behavior of the synthesized kernel at clock cycle level. Although the simulator models the generated FPGA datapath, the external components such as memory are simplified. Therefore, the latency may slightly differ from physical hardware. This stage takes longer than the emulation stage; in the work covered in this paper, the simulation took around 30 minutes.

Once the design has been functionally verified through emulation and its generated hardware behavior has been evaluated through simulation, the final stage is compiling and executing for the physical FPGA hardware. Compiling for this stage requires the Quartus Prime software. Due to constraints in the availability of the hardware within this project timeline, The project in this paper has not yet reached this stage. According to intel's documentation, compilation for FPGA hardware takes hours to complete.

3.2.2 Alpaka Implementation

Alpaka provides an abstraction layer through which the same kernel implementation can target different backends. Therefore, the compilation and execution process depends on the backend selected. In this work, SYCL is selected as the backend; therefore, the FPGA compilation stages are handled through the Intel oneAPI SYCL toolchain described in the previous section [5].

The work presented in this paper successfully compiles and executes Alpaka CLUE implementation for FPGA emulation. However, the subsequent stages of simulation and hardware execution have not yet been completed.

3.3 Reference Implementations

This project builds on existing implementations of the CLUE clustering algorithm. The implementations were used as a starting point and as references for verifying outputs.

The SYCL implementation was inherited from previous OpenLab work¹. The implementation was already FPGA oriented, however, it was not fully optimized for CLUE. From this baseline, several versions were developed to reach a fully optimized implementation using Intel oneAPI SYCL. The modifications and their results are discussed in sections 4 and 6.1.

The Alpaka implementation is based on the existing CLUE implementation using the Alpaka library². This version was not originally designed for FPGA execution. It was used as a starting point for investigating whether CLUE could be adapted to target FPGAs through Alpaka. This implementation targeting FPGA is discussed in sections 5 and 6.2.

3.4 Datasets

The implementations were primarily tested using the anisotropic, *aniso_1000*, dataset containing 1000 detector hits. Additional datasets were also tested during development to verify correct behavior. All datasets were obtained from the reference implementations of CLUE.

¹Baseline SYCL implementation of CLUE <https://github.com/MohamadKhaledCharaf/CLUE-SYCL.git>

²Reference Alpaka implementation of CLUE <https://github.com/cms-patatrack/CLUEstering.git>

4 SYCL FPGA CLUE Implementation³

Section 4 presents the SYCL FPGA implementation used as the baseline, the modification done to address the performance bottlenecks, and pipelining and parallelization strategies implemented.

4.1 Implementation

In the original SYCL code, first the input data is copied from CPU host memory to the FPGA's board internal memory then the CLUE algorithm is broken into five separate `single_task` kernels:

1. `ComputeHistogram`: builds the fixed-grid spatial index.
2. `CalculateDensity`: iterates through all hits to compute the ρ values.
3. `CalculateDistanceToHigher`: iterates through all hits to compute the δ values.
4. `FindClusters`: identify seeds and outliers and construct follower relationships.
5. `AssignClusters`: Traverses follower relationships to propagate cluster IDs.

The kernels are executed sequentially, then the final cluster results are copied from the FPGA back to the host.

Loops inside kernels can be implemented as hardware pipelines to allow iterations to overlap. Latency is the number of clock cycles required for one iteration to pass through the pipeline. The initiation interval (II) represents the frequency of new iterations entering the pipeline in clock cycles. Performance does not depend on latency of an individual iteration solely but also on II.

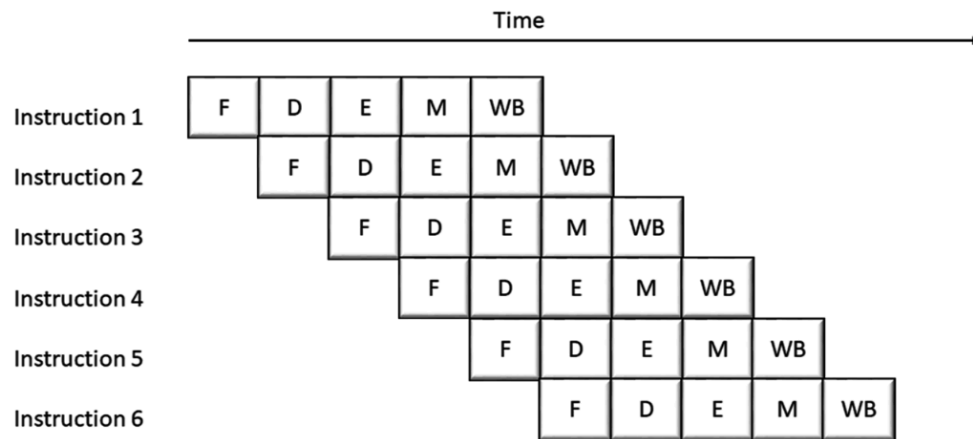


Fig. 3. Illustration of pipelined instruction processing. F is fetch, D is decode, E is execute, M is memory, WB is write-back [3].

Pipelined processes allow different iterations to occupy different stages of the hardware at the same time, illustrated in Fig. 3. The latency is the number of clock cycles one iteration takes to pass through all pipelined stages, assuming each stage takes one clock cycle, the latency in this

³ CLUE FPGA SYCL & Alpaka implementations <https://github.com/Ghqlq/CLUE-FPGA-Porting-CERN.git>

example is 10 clock cycles. In case of loop-carried dependencies, II would increase because a new iteration may need to wait until a result required from a previous iteration is available.

Since the original SYCL implementation was used as a starting point for the work in this paper, it was important to look at the performance bottlenecks resulting from this design in order to address those challenges. Two performance bottlenecks identified in the previous work are caused by the `ComputeHistogram` kernel and the `AssignCluster` kernel.

4.1.1 Histogram Modification

The `ComputeHistogram` kernel assigns every hit to a tile in the fixed spatial grid. For each hit, `fill()` function determines its tile then stores the hit index at the current position in that tile:

```
void fill(float x, float y, int i, int *tileSizes) {
    int tileIndex = getGlobalBin(x, y);
    tiles_[tileIndex].push_back(i, tileSizes[tileIndex]);
}
```

However, for the histogram to keep track of the number of hits stored in each tile it needs to increment `tileSizes`:

```
++tileSizes[tileIndex];
```

This introduces a loop-carried read-afterwrite dependency in the `ComputeHistogram` kernel since the position used by the current hit depends on the value written by a previous iteration. This dependency limits the ability to pipeline the loop and achieve a low initiation interval.

The modified implementation reduces the cost of this dependency by moving the tile counters to an `OnchipMemoryWithCache` [7].

```
fpga_tools::OnchipMemoryWithCache<int,
                                NLAYERS * T::nTiles,15> tileSizes(0);
```

Then the kernel updates the number of hits stored in each histogram tile, which retains the same dependency faced with the previous implementation:

```
int count = tileSizes.read(combinedIndex);

if (count < T::maxTileDepth) {
    hist_d[layeri].fill(i, tileIndex, count);
    count++;
    tileSizes.write(combinedIndex, count);
}
```

The on-chip cache does not remove the dependency since `count(i+1)` still depends on `count(i)` when the same tile is hit again. However, the updated state of the counter is acquired from an on-chip memory instead of requiring the longer memory path each iteration. After all hits are processed, the final cached counts are copied back into the histogram.

4.1.2 Cluster Assignment Modification

The original code builds a follower list for each hit in `FindClusters` kernel, then traverses through the follower tree from every seed using Depth-First Search (DFS) to assign cluster indices while repeatedly pushing and popping hits from a local stack in `AssignCluster` kernel. This creates a loop-carried memory dependency, making it not ideal for pipelining.

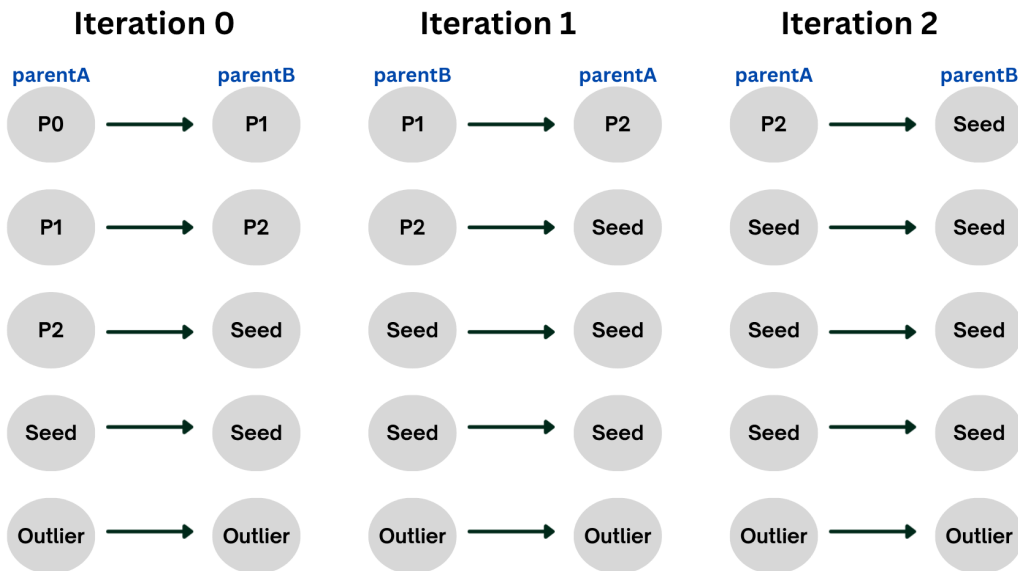


Fig. 4. Parent Propagation with Alternating Buffers, `parentA` and `parentB`.

The modified implementation stores each point's nearest-higher point as its parent in `FindClusters` kernel. Therefore, instead of storing the children of each hit, each non-seed hit stores a single reference to its nearest-higher hit referred to as parent in this paper. Seeds point to themselves while outliers also point to themselves but get assigned a cluster index of -1. This mapping of points to parents is stored in an array, `parentA`. The `AssignClusters` kernel was renamed to be `AssignParents`. The alternating buffer parent propagation approach is illustrated in Fig. 4.

`AssignParents` uses pointer jumping to alternate between two buffers, `parentA` and `parentB`. In one iteration, all reads refer to the unchanged parent mapping from the previous iteration while updates are written into the other buffer. The roles of the two buffers are then exchanged for the next iteration. This loop continues until one of the buffers last written into is

unchanged from the previous iteration. This indicated that points have reached their seeds. The structured passes over the two point arrays make it suitable for FPGA optimization.

4.2 Pipelining and Parallelization

After the first modified version of the CLUE implementation described above, two versions of the implementation were investigated: a pipelined version and a parallelized version.

The pipelined version increases throughput by enforcing loop iterations to overlap. This is done through instructing the compiler to ignore assumed dependencies using an Intel FPGA compiler attribute, `ivep`, which stands for ignore vector dependencies.

This strategy was used in the histogram kernel and the cluster assignment kernel. As a result, a new iteration can enter the pipeline every clock cycle.

The parallelized version builds on the same structure as the pipelined version but processes multiple points within the loop concurrently. In the case of the histogram kernel, the tile processing loops are divided into fixed-width groups and the operations within each group are unrolled to create multiple parallel hardware lanes. This is done through `unroll` pragma instructions.

5 Alpaka FPGA CLUE Adaptation⁴

The Alpaka implementation of CLUE was developed by adapting the FPGA-oriented SYCL version of CLUE described in Section 4. This implementation uses the same five sequential kernels: `ComputeHistogram`, `CalculateDensity`, `CalculateDistanceToHigher`, `FindClusters`, and `AssignParents`. However, the SYCL-specific device management, memory allocation, kernel definitions, and kernel submissions were replaced with the corresponding Alpaka abstractions.

5.1 Implementation

The CLUE implementation using Alpaka targeting CPU and GPU execution distributes the work within a kernel over multiple threads, allowing for parallelism. However, this prevents pipelining in an FPGA.

In the Alpaka implementation device memory is managed using buffer objects, which are owned by the `DeviceBuf` structure. Native pointers that are used inside the kernels are obtained using `alpaka::getPtrNative()`. Those pointers are used in a similar way to device pointers in the SYCL implementation. Host-to-device and device-to-host communication are also rewritten in the Alpaka implementation. In Alpaka, `alpaka::ViewPlainPtr` is used to represent CPU arrays.

⁴ CLUE FPGA SYCL & Alpaka implementations <https://github.com/Ghqlq/CLUE-FPGA-Porting-CERN.git>

Copying data from the host to Alpaka device buffer is done using `alpaka::memcpy`. Each kernel is created using `alpaka::createTaskKernel` and later enqueued into the Alpaka queue. This preserved the sequential dependency in the CLUE algorithm.

The computational structure within each kernel was kept similar to the original SYCL implementation of CLUE targeting FPGA. Additionally, the cluster assignment modification introduced in Section 4 in the modified SYCL implementation was retained.

The modification to the `ComputeHistogram` kernel discussed in Section 4 and implemented in the SYCL versions could not be retained in the Alpaka implementation because the `OnChipMemoryWithCache` abstraction is specific to the Intel FPGA SYCL toolchain. Therefore, the original approach to construct histogram was reintroduced here. In this approach, the tile occupancy counters are updated directly during histogram filling. This approach is known to have a loop-carried memory dependency due to the read-after-write operation in the tile counters.

To reproduce the execution of the SYCL FPGA implementation using Alpaka, work division was configured with one block, one thread per block, and one element per thread. Therefore, there was no distributed work across threads unlike the Alpaka GPU implementation of CLUE. This is equivalent to the `single_task` approach.

```
alpaka::Vec<Dim, Idx> const threadsPerBlock(static_cast<Idx>(1));
alpaka::Vec<Dim, Idx> const blocksPerGrid(static_cast<Idx>(1));
alpaka::Vec<Dim, Idx> const elementsPerThread(static_cast<Idx>(1));

using WorkDiv = alpaka::WorkDivMembers<Dim, Idx>;
auto const manualWorkDiv =
    WorkDiv{blocksPerGrid, threadsPerBlock, elementsPerThread};
```

`ComputeLocalDensity` and `ComputeDistanceToHigher` kernels retain the same main calculations and spatial searches as the original Alpaka implementation, and `FindClusters` retains the same seed and outlier conditions. The changes were therefore concentrated on the sections where the original GPU execution structure conflicted most strongly with an FPGA implementation: histogram counter updates and the irregular follower-tree traversal used for cluster propagation.

6 Results

The results of the project have two dimensions. The first one was functional correctness, where results from the modified implementations were compared with those of the reference implementations of the CLUE algorithm. The second was performance, as the algorithms were optimized to achieve $\Pi = 1$, as well as increase the throughput of the algorithms to the maximum.

An implementation can be correct on the emulator, but its performance can be poor when tested on the simulator.

For functional correctness, algorithms were tested on the emulator. Since the emulator takes seconds to compile, this allows quick debugging of the code.

To test for performance, the program was built and compiled for the simulator. Although the simulator did not run on large datasets, the compiler reports were useful to examine multiple performance metrics. Metrics included Memory dependencies, estimated resource usage, II, and the estimated maximum clock frequency.

6.1 SYCL-Based Implementation Results

6.1.1 Results Verification

The CLUE implementation using Alpaka targeting CPU and GPU execution distributes the work within a kernel over multiple threads, allowing for parallelism. However, this prevents pipelining in an FPGA.

All SYCL implementations presented in this paper were compiled and executed for the Intel FPGA emulator. Outputs from all versions matched the reference results from the original CLUE implementations targeting GPU.

Although the emulator checks for correctness, there were conclusions drawn on performance because the emulator does not target a specific FPGA hardware.

6.1.2 Performance on FPGA

After verifying correctness, SYCL implementation was compiled and executed for the simulator targeting a DE10-Agilex environment.

The original SYCL FPGA implementation, the compiler report revealed two important pipeline bottlenecks. `ComputeHistogram` contained a memory dependency that limited one critical loop to $II = 2$. The original cluster assignment implementation also contained loops that could not be pipelined effectively because the compiler had to be explicitly told. However, the primary loops in `CalculateDensity` and `CalculateDistanceToHigher` were already able to operate at $II = 1$.

After optimization, all loops for which pipelining was considered feasible reached $II = 1$. This represented the main throughput objective of the direct SYCL optimization.

TABLE 1
Compute Histogram Performance Analysis

Version	II	Estimated f_{MAX} (MHz)	Latency (Cycles)
Original	2	-	-
First Optimization	2	480	389
Final Optimization	1	255	424

This result illustrates why block latency solely is not sufficient to judge an FPGA optimization. The final histogram block has a slightly larger reported latency in cycles than the first optimized version. However, the II improved. Additionally, the estimated maximum clock frequency, f_{MAX} , decreases. For a conclusion about the estimated maximum clock frequency, a hardware run is required.

TABLE 1
Cluster Assignment Kernel Performance Analysis

Version	II	Estimated f_{MAX} (MHz)	Latency (Cycles)
Original	N/A	-	-
First Optimization	493	480	816
Final Optimization	1	480	822

The cluster assignment kernel improvement is more obvious. Originally, the kernel was not pipelined. After pipelining the kernel, the first optimized parent assignment loop reported an II of 493. After the dependency structure was redesigned coupled with enforcing pipelining, the loop achieved II = 1, with a similar latency.

It is also important to examine the resources utilized in order to determine whether the design can fit on the target FPGA, otherwise to identify potential resource bottlenecks. Both final versions, pipelined and parallelized, did not come close to exhausting the FPGA resource.

6.2 Alpaka FPGA CLUE Adaptation Results

6.2.1 Results Verification

The Alpaka implementation was verified for output correctness as well. Because Alpaka adds a top layer of abstraction, this implementation used Cmake to configure the build then compiled through Alpaka SYCL backend. The results were compared to those of its reference implementation targeting GPU hardware.

Due to time constraints and the additional challenge posed by the dependency discussed in Section 5.1 concerning kernel `ComputeHistogram`, Alpaka implementation could not be compiled for simulation.

7 Conclusions

7.1 Challenges and Limitations

One main challenge was not having access to the target FPGA hardware during the early stages of the project. As a result, for most of the duration of the project, testing and development had to rely on FPGA emulation rather than target-specific testing. Access to an FPGA became available in the last couple of weeks of the project; however, due to the limited time frame, execution on the physical FPGA could not be completed.

As a result of delayed hardware access, the final implementations could not be validated on the target hardware. Therefore, measured metrics, including execution time, throughput, and power consumption could not be obtained.

7.2 Conclusion

This project investigated the porting and optimization of the CLUE reconstruction algorithm for FPGA execution using Intel oneAPI and Alpaka.

The SYCL implementation used a prior SYCL FPGA implementation as a baseline to optimize. The main performance limitations were in two kernels, `ComputeHistogram` and `AssignClusters`. Both kernels contained a loop-carried memory dependency. The `ComputeHistogram` kernel had a read-after-write dependency that was mitigated by implementing an on chip memory with cache. The `AssignClusters` kernel used a depth first search which created irregular access not suitable for pipelining. This approach was replaced by two buffers alternating for read and write jobs.

The final SYCL implementation utilized intel instruction that enforces pipelining and achieved an initiation interval of one across all kernels. The implementation remained within the available resources of the target FPGA.

The Alpaka implementation retained most of the approaches used in the final version of the SYCL implementation. However, the on chip memory with cache is not supported by Alpaka and could not be retained. Consequently, the original approach that is known to have loop-carried dependency was reintroduced.

This work provides a step forward toward optimizing CLUE implementation on FPGAs using Intel oneAPI SYCL as well as an initial basis for porting the CLUE algorithm on FPGAs using Alpaka.

7.3 Future Work

Final SYCL FPGA implementation of CLUE, both pipelined and parallelized, should be compared on hardware. This would determine whether additional hardware resources used by unrolling in the parallelized version have significant improvement compared to the pipelined implementation.

Further work is required for the Alpaka implementation to be compiled for simulation. Specifically, the dependency discussed in Section 5.1 on kernel `ComputeHistogram`.

A comparison between the SYCL and the Alpaka implementation would be a future work after both implementations are executed on hardware. Implementations should also be tested on larger datasets to provide a more complete assessment of how well they scale.

7.4 Acknowledgements

I would like to thank my supervisors, Andrea Bocci and Mario Gonzalez Carpintero, for their guidance. I would also acknowledge the members of the CMS collaboration and colleagues for their useful discussions. I am grateful for the CERN Summer Student Programme for providing the opportunity to carry out this project.

8 References

- [1] CMS Collaboration, *The Phase-2 Upgrade of the CMS Endcap Calorimeter: Technical Design Report*, CERN-LHCC-2017-023, CMS-TDR-019, CERN, Geneva, Switzerland, 2018.
- [2] M. Rovere, Z. Chen, A. Di Pilato, F. Pantaleo, and C. Seez, “CLUE: A fast parallel clustering algorithm for high granularity calorimeters in high-energy physics,” *Frontiers in Big Data*, vol. 3, Art. no. 591315, 2020, doi: 10.3389/fdata.2020.591315.
- [3] A. Chattopadhyay, Ed., *Handbook of Computer Architecture*. Singapore: Springer Nature Singapore, 2025, doi: 10.1007/978-981-97-9314-3.
- [4] Intel Corporation, *Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs*. Intel Corporation, 2025.
- [5] Alpaka Group, “alpaka — An abstraction library for parallel kernel acceleration,” *alpaka Documentation*. <https://alpaka.readthedocs.io/en/latest/index.html>.
- [6] Terasic Inc., “DE10-Agilex Development Kit,” *Terasic Technologies Inc.* <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=142&No=1252&PartNo=2>.
- [7] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook, and X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2nd ed. Berkeley, CA, USA: Apress, 2023, doi: 10.1007/978-1-4842-9691-2.
- [8] M. K. Charaf, A. Bocci, and A. Perego, “Exploring FPGA acceleration for CMS reconstruction using alpaka,” Zenodo, Oct. 18, 2025, doi: 10.5281/zenodo.17383508. <https://zenodo.org/records/17383508>